Angewandte KI & KI-Engineering (AKE)

Block A – Vertiefung Deep Learning

Lab 2: CNNs & Computer Vision

Vom Filter zum Trägermodell in fünf Schritten

Wintersemester 2026/27

Organisatorisches – Lab Woche 2

Termin	KW 42 (15. 10. 2026), 2-SWS-Lab-Block im Anschluss an den 4-SWS-Theorieblock; 90 Min. Präsenz
Skript-Bezug	*CNNs & Computer Vision*, Abschn. 2–3 (Faltung, Pooling, LeNet), Abschn. 5 (Transfer Learning), Abschn. 8 (Fehleranalyse); Trainingsschleife und MLP aus *Vom Perzeptron zum Deep Network* (Lab)
Material	Kursrepository `lab02/`: dieses Aufgabenblatt, `lab02_starter.ipynb`, `results_referenz/` (Referenzlogs des Lösungslaufs); `src/traeger.py` (Datensatz, Vorverarbeitung, Modellbau); `lab02_loesung.ipynb` wird nach der Sitzung freigeschaltet
Umgebung	Kursumgebung aus dem ersten Lab (`git pull`, danach `uv sync`); es wird nichts neu installiert
Daten	Im Repository unter `data/`: `MNIST/`, `imagenette5/` (5 Klassen, 2 500 Trainings-/500 Validierungsbilder, 160 px, feste Dateilisten `train.txt`/`val.txt`, Klassenreihenfolge in `classes.json`), `models/resnet18-f37072fd.pth` (ImageNet-Gewichte, 45 MB)
Zeitplan	0–5 Setup (Aufgabe 0) · 5–20 Aufgabe 1 · 20–30 Aufgabe 2 · 30–45 Aufgabe 3 · 45–70 Aufgabe 4 · 70–85 Aufgabe 5 · 85–90 Puffer. Reine Rechenzeit ≈ 5 Min. (4 Kerne); Optionales als Selbststudium

Worum es geht

Sie bauen die Konzepte der Vorlesung in kleinen Schritten selbst nach – erst mit NumPy auf Papiergröße, dann mit PyTorch auf echten Bildern. Am Ende steht das **Trägermodell**: ein ResNet-18, per Transfer Learning auf fünf Imagenette-Klassen angepasst, das im weiteren Semester versioniert, deployt, überwacht und erklärt wird. Das Starter-Notebook gibt alles vor, was nicht zum Lernziel gehört (Daten laden, Trainingsschleife, Plots). Sie schreiben nur die Kernzeilen.

Aufgaben

Aufgabe 0: Setup (5 Min., Pflicht, ohne Abgabe)

1. Im Ordner `ake`: `git pull`, dann `uv sync`; `uv run jupyter notebook` starten.
2. `lab02/lab02_starter.ipynb` als `lab02_<nachname>.ipynb` kopieren und nur in der Kopie arbeiten.
3. Setup-Zelle ausführen: Sie prüft MNIST, Imagenette5 (2 500/500 Bilder, 5 Klassen) und die ResNet-18-Gewichte und legt `results/`, `cache/` und `models/` an. Erwartung: alle Prüfungen „OK“.

Aufgabe 1: Die Faltung von Hand (15 Min.)

Skript-Bezug: Def. 2.1, Abb. 2, Bsp. 2.2, Satz 2.3

Ein Filter ist ein Musterdetektor: An jeder Position wird das Skalarprodukt aus Kernel und gleich großem Bildausschnitt gebildet.

1. `conv2d_np(I, K)` als Doppelschleife nach Gl. (1), ohne Padding, Stride 1. Testfall ist Abb. 2:

$$\boldsymbol{I} = \begin{pmatrix} 2 & 5 & -3 & 0 \\ 0 & 6 & 0 & -4 \\ -1 & -3 & 0 & 2 \\ 5 & 0 & 0 & 3 \end{pmatrix}, \quad \boldsymbol{K} = \begin{pmatrix} -2 & 0 \\ 4 & 6 \end{pmatrix} \Rightarrow \boldsymbol{C} = \begin{pmatrix} 32 & 14 & -18 \\ -22 & -24 & 12 \\ 22 & 6 & 18 \end{pmatrix}.$$

2. Um Padding P (`np.pad`) und Stride S erweitern; `out_size(J, M, P, S)` nach Gl. (2). Prüfen: $(28, 5, 2, 1) \to 28$, $(224, 7, 3, 2) \to 112$; Abb. 2 mit P=1, S=2 ergibt 3×3.
3. Dieselbe Rechnung mit `F.conv2d`. Formen Sie $\boldsymbol{I}$ und $\boldsymbol{K}$ dazu in das PyTorch-Format (N, C, H, W) um, also $(1, 1, 4, 4)$ und $(1, 1, 2, 2)$, und vergleichen Sie mit `torch.allclose`.
4. $\boldsymbol{K}_{\text{vert}}$ aus Bsp. 2.2 und $\boldsymbol{K}_{\text{hor}} = \boldsymbol{K}_{\text{vert}}^{\top}$ mit Ihrer Funktion auf das synthetische Testbild und auf eine MNIST-Ziffer anwenden; die Plotfunktion ist vorgegeben.

Warum ist die Filterantwort auf homogenen Flächen null, und was codiert ihr Vorzeichen? $\boldsymbol{K}_{\text{vert}}$ hat 9 Gewichte. Eine vollverbundene Schicht, die aus einem 28×28-Bild eine gleich große Ausgabe berechnet, hätte $784 \cdot 784 = 614\,656$. Welche zwei Ideen aus Abschn. 2.2 erklären den Unterschied?

Aufgabe 2: Bausteine zählen – Parameter, Pooling, Formen (10 Min.)

Skript-Bezug: Satz 2.4, Listing 1, Def. 3.1, Tab. 2, Listing 2

1. `n_params_conv(M, C_in, C_out)` nach Gl. (3); Vergleich mit `sum(p.numel() ...)` für `nn.Conv2d(3, 64, 3)` (erwartet 1 792, Bsp. 2.5) und für den Block aus Listing 1 (Conv ohne Bias + BatchNorm, erwartet 1 856).
2. Max- und Average-Pooling (2×2, Stride 2) von $\boldsymbol{F} = \begin{pmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 1 \\ 0 & 2 & 5 & 7 \\ 1 & 1 & 3 & 2 \end{pmatrix}$ von Hand, dann mit `F.max_pool2d` und `F.avg_pool2d` prüfen.
3. In der Klasse `LeNet` (Listing 2) die Eingangsgröße der ersten linearen Schicht ergänzen. `torch.zeros(1,1,28,28)` Schicht für Schicht durch `model.features` schicken, jede Form ausgeben und mit Tab. 2 vergleichen. Erwartung: $16 \times 5 \times 5$ vor dem Flatten, 61 706 Parameter.

Warum werden die Feature Maps von Stufe zu Stufe kleiner, während die Kanalzahl wächst? Warum hat Pooling keine Parameter? In welcher Schicht von LeNet stecken die meisten Parameter (Tab. 2), und was gewinnt man deshalb mit Global Average Pooling (Abschn. 3.1)?

Aufgabe 3: CNN gegen MLP auf MNIST (15 Min., Rechenzeit ≈ 1 Min.)

Skript-Bezug: Abschn. 1.1, 2.2, 3.1, 3.3

1. LeNet und MLP 784–128–128–10 mit `fit()` aus dem ersten Lab trainieren: Adam 10^{-3}, Batch 64, **2 Epochen**, `torch.manual_seed(0)` vor jedem Modell; Testsplit genau einmal auswerten. Tabelle: Testgenauigkeit, Parameterzahl, Zeit pro Epoche. Erwartung: LeNet ≥ 98,5 %, MLP ≈ 96–97 %.
2. Verschiebungstest: alle Testbilder mit `torch.roll(x, shifts=s, dims=3)` um $s \in \{0, 1, 2, 3, 4, 6\}$ Pixel nach rechts schieben und beide Modelle erneut messen; Genauigkeit über s plotten. Erwartung bei $s = 3$: LeNet ≈ 90 %, MLP ≈ 55–60 % (Robustheitsdiagnose, keine Modellauswahl).

LeNet hat etwa halb so viele Parameter wie das MLP. Woher kommt sein Vorsprung? Unterscheiden Sie Äquivarianz (Faltung, Abschn. 2.2) und Invarianz (Pooling, Abschn. 3.1): Welche der beiden Eigenschaften zeigt der Verschiebungstest? Warum bricht auch LeNet bei 6 px ein? Warum ist LeNet pro Epoche trotzdem langsamer (Satz 2.4, Bsp. 2.5)?

Aufgabe 4: Transfer Learning – das Trägermodell entsteht (25 Min., Rechenzeit ≈ 4 Min.)

Skript-Bezug: Def. 5.1, Algorithmus 1, Listing 3, Abschn. 5.4

1. *Modellchirurgie.* ResNet-18 mit den ImageNet-Gewichten aus `data/models` laden, `print(model.fc)` ausgeben (erwartet `Linear(512, 1000)`). Den Kopf durch `nn.Linear(512, 5)` ersetzen, alle übrigen Parameter einfrieren, trainierbare Parameter zählen. Erwartung: 2 565 von 11 179 077.
2. *Features cachen.* `extract_features()` berechnet $h(\boldsymbol{x})$ (Gl. 5) für alle 3 000 Bilder genau einmal und legt sie in `cache/` ab (≈ 1 Min.).
3. *Feature-Extraktion.* Linearen Kopf auf den gecachten Features mit Adam trainieren, Lernrate $\epsilon \in \{10^{-4}, 10^{-3}, 10^{-2}\}$, je 10 Epochen (Sekunden pro Lauf). Validierungsgenauigkeit aller drei Läufe in *einem* Diagramm; Kopf mit minimalem Validierungsverlust übernehmen. Erwartung: ≥ 95 %.
4. *Partielles Fine-Tuning.* `layer4` freischalten (erwartet 8 396 293 trainierbare Parameter); zwei Parametergruppen (Kopf 10^{-3}, `layer4` 10^{-4}); BatchNorm über die vorgegebene `train_mode()` im `eval()`-Modus; Augmentierung (Abschn. 3.4); **2 Epochen** (≈ 2–3 Min.). Erwartung: ≥ 95 % (Ziel des Trägermodells: ≥ 90 %).
5. *Ablage.* `state_dict` als `models/traeger_v1.pt`; Vorlage `models/traeger_v1.yaml` ergänzen (Klassenliste, Vorverarbeitung, Seed, Hyperparameter, Validierungsgenauigkeit). Ladetest: Modell mit `load_traeger()` *nur aus der YAML-Datei* neu aufbauen und ein Validierungsbild klassifizieren. Erwartung: gleiche Vorhersage wie das trainierte Modell.

Nach dem Kopftausch sind nur 0,02 % der Parameter trainierbar. Warum reicht das? Warum konvergiert der lineare Kopf so schnell (Def. 5.1)? Das Skript erwartet für die Feature-Extraktion 85–90 %, Sie messen deutlich mehr. Was ist an diesen fünf Klassen besonders (Tab. 6, Domänenabstand)? Warum bekommt `layer4` eine zehnmal kleinere Lernrate als der Kopf? Warum gehören Klassenreihenfolge und Normalisierung in die YAML-Datei statt ins Notebook?

Aufgabe 5: Fehleranalyse des Trägermodells (15 Min.)

Skript-Bezug: Abschn. 8.1–8.2, Gl. (7), (8), Analyseprotokoll

1. Konfusionsmatrix $\boldsymbol{M} \in \mathbb{N}^{5\times5}$ selbst berechnen: für jedes Validierungsbild mit wahrer Klasse i und Vorhersage j gilt `M[i, j] += 1`. Die Heatmap ist vorgegeben. Genauigkeit = $\operatorname{tr} \boldsymbol{M} / \sum_{ij} M_{ij}$.
2. Precision_j (Spalte) und Recall_i (Zeile) nach Gl. (7), (8) direkt aus $\boldsymbol{M}$; Gegenprobe mit `sklearn.metrics.classification_report`.
3. Die fünf überzeugtesten Fehler anzeigen, also die Bilder mit der höchsten Softmax-Wahrscheinlichkeit für die *falsche* Klasse, mit wahrer Klasse, Vorhersage und Konfidenz.
4. Jeden dieser Fehler einer Kategorie aus Abschn. 8.2 zuordnen (Label-Fehler, Mehrdeutigkeit, Kurzschluss über Kontext, Verteilungsverschiebung, Kapazität) und *eine* Maßnahme samt erwarteter Wirkung ableiten, ohne sie umzusetzen.

Sind die Fehler gleichverteilt oder auf wenige Paare konzentriert? Wie sicher ist das Modell bei seinen Fehlern, und wofür ließe sich das im späteren Service nutzen? Bei rund zehn Fehlern auf 500 Bildern: Wie belastbar ist eine Aussage über das „häufigste Verwechslungspaar“?

Aufgabe O: Optional (Selbststudium)

O1 *IoU (Def. 6.1).* `iou(a, b)` für Boxen (x_1, y_1, x_2, y_2). Tests: identische Boxen $\rightarrow$ 1; disjunkte $\rightarrow$ 0; $(0,0,2,2)$ vs. $(1,1,3,3)$ $\rightarrow$ $1/7 \approx 0{,}143$; $(0,0,4,4)$ vs. $(1,1,3,3)$ $\rightarrow$ 0,25. Wie viele Zahlen gibt YOLO für S=7, B=2, C=20 aus (Def. 6.2)?

O2 *Grad-CAM (Gl. 9).* Für die drei überzeugtesten Fehler aus Aufgabe 5 mit `pytorch-grad-cam` auf `layer4`, je für vorhergesagte *und* wahre Klasse: Hat das Netz auf das Objekt oder auf den Hintergrund geschaut?

O3 *Normalisierung weglassen.* Features ohne ImageNet-Normalisierung neu extrahieren, gleichen Kopf trainieren und vergleichen. Das ist Data Drift im Kleinen (Abschn. 5.2).

O4 *im2col.* Die Faltung aus Aufgabe 1 als eine Matrixmultiplikation schreiben und die Laufzeit gegen die Doppelschleife messen.

Hinweise

In der Praxis

Erst Formen prüfen, dann trainieren. Der häufigste Fehler ist eine falsche Eingangsgröße der ersten linearen Schicht: Gl. (2) nachrechnen oder die Form ausgeben lassen. PyTorch erwartet (N, C, H, W); ein mit PIL geladenes Bild liegt als (H, W, C) vor. **Features cachen:** Bei eingefrorenem Backbone hängt $h(\boldsymbol{x})$ nicht von den trainierbaren Parametern ab. Einmal rechnen, dann kostet jede Kopf-Epoche Millisekunden.

- **Reproduzierbarkeit:** `torch.manual_seed(0)` vor *jedem* Modell; jedes Experiment schreibt sein JSON-Log nach `results/`. Klassenreihenfolge und Splits nur aus `classes.json`/`train.txt`/`val.txt`, nie aus `os.listdir`.
- **Evaluation:** immer mit `model.eval()` und `torch.no_grad()`. Der MNIST-Testsplit wird je Modell genau einmal zur Bewertung genutzt. Imagenette5 hat nur Trainings- und Validierungssplit; weil die Validierungsmenge der Modellauswahl dient, ist die Genauigkeit aus Aufgabe 5 optimistisch.
- **Fallback auf Referenzlogs:** Dauert ein Lauf zu lange oder bricht er ab, laden Sie das passende JSON aus `results_referenz/` und bearbeiten die Interpretation damit; markieren Sie das im Notebook.
- **Trägermodell:** `models/traeger_v1.pt` und `traeger_v1.yaml` nicht umbenennen oder verschieben – ab dem Skript *Daten-Engineering für ML* bauen alle Labs auf genau diesen Dateien auf.